FABULOUS
=
F# FUNCTIONAL FOR YOUR APP

Don Syme, Microsoft Researcher, F# Contributor

+ assistance from Mark Allibone, Jim Bennett, Jason Imison, Miguel de Icaza, Phillip Carter, Fabulous Community Contributors

THANKS

- Timothé Larivière also Zaid-Ajaj
  - co-maintainer on Fabulous, author of ElmishContacts and ElmishPlanets demos
- Jim Bennett
  - Great talks on Fabulous!
- Mark Allibone
  - Swiss MVP , @mallibone
- Frank Krueger
  - ImmutableUI was inspiration and initial partial list of Xamarin.Core bindings
- Boris D. (github @dboris)
  - authored "Elmish.Forms" ("half-elmish" for static views only)
- Jason Imison
  - F# in Visual Studio for Mac and Xamarin
- Eugene Tolmachev
  - Creator or "Elmish" for Fable, used for WPF and others
- Justin Sacks and others
  - Creators of "Elmish.WPF" ("half-elmish" for static views only)
- Alfonso, Maxime, Steffen and many others
  - The Fable gang








DYLAN AND THE LINEBREAKERS


We're Gonna Build a
Framework!
A song about code by @dylanbeattie
Music "We Didn't Start the Fire" by Billy Joel
© 1989 Columbia Records

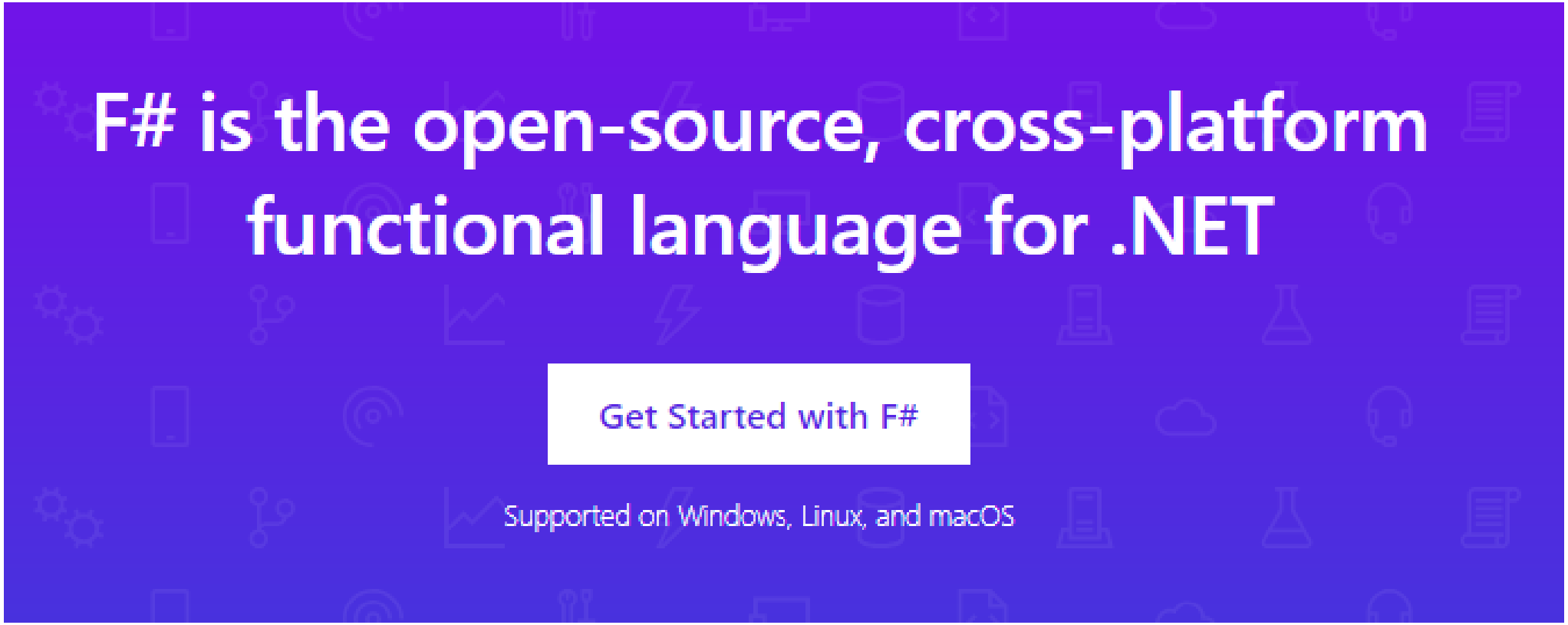
First, a little about F#

Aside

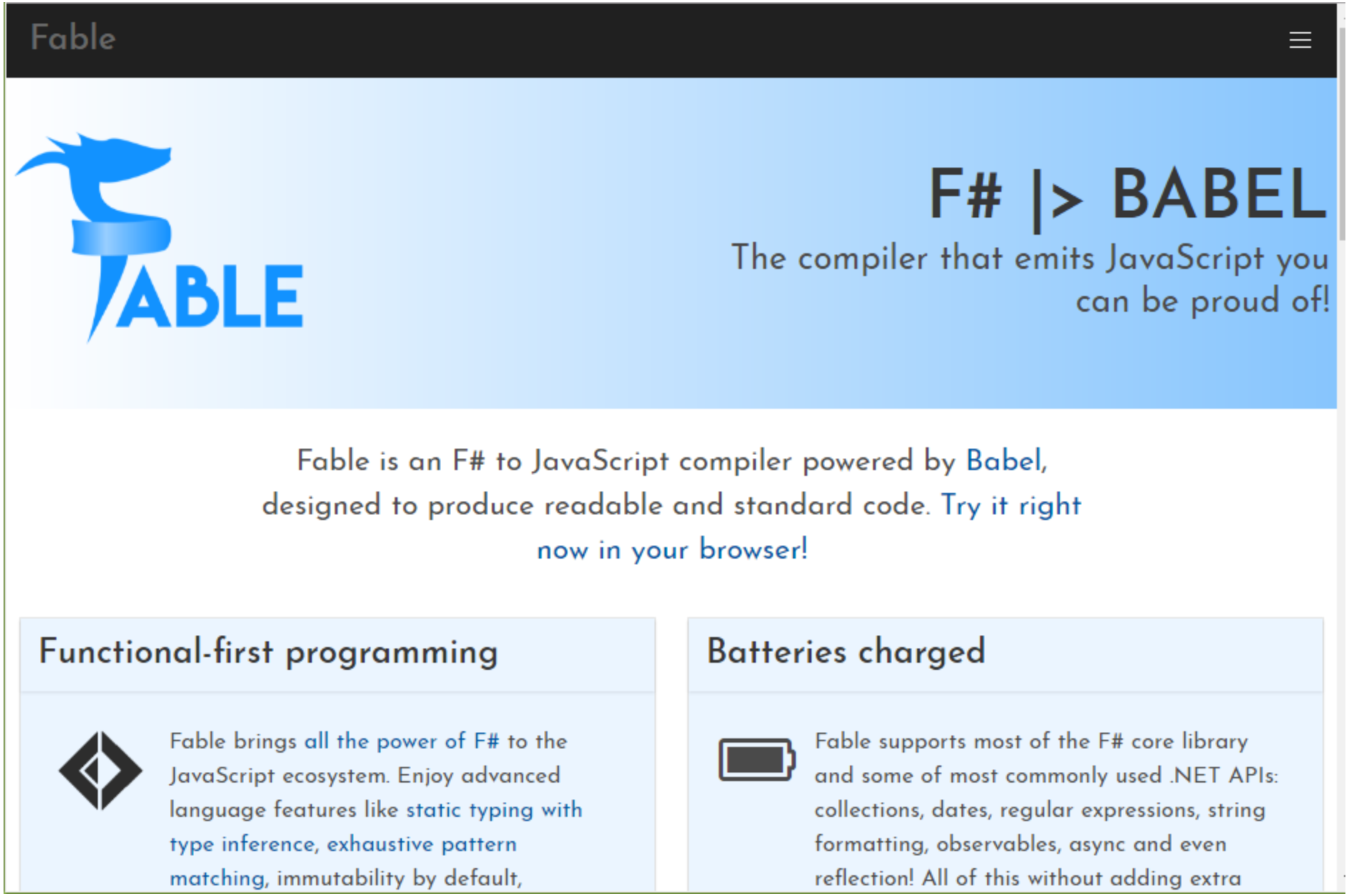
The Early History of F# - HOPL IV submission (2020)

Draft now available!

fsharp.org/history



www.microsoft.com/net/


Fable
F# |> BABEL
The compiler that emits JavaScript you can be proud of!
Fable is an F# to JavaScript compiler powered by Babel, designed to produce readable and standard code. Try it right now in your browser!
Functional-first programming
Fable brings all the power of F# to the JavaScript ecosystem. Enjoy advanced language features like static typing with type inference, exhaustive pattern matching, immutability by default,
Batteries charged
Fable supports most of the F# core library and some of most commonly used .NET APIs: collections, dates, regular expressions, string formatting, observables, async and even reflection! All of this without adding extra

TL;DR – Where we are now

F# is open, cross-platform, neutral, independent

All F# language and tooling is accepting contributions

The F# community is self-empowered

fsharp.org

Xamarin tools for Android, iOS, macOS etc.

VSCode, VS for Windows, Mac

.NET Core for Linux/OSX

The F# Compiler Service powers other tools

Fable for F#-to-Javascript

SAFE for F# fullstack

F# 4.6 now complete!

The F# Language Design Process

github.com/fsharp/fslang-design
github.com/fsharp/fslang-suggestions

F# 4.0

- ✓ True shift to cross-platform open engineering
- ✓ Long laundry list of language items
- ✓ Normalized core library
- ✓ Type providers more powerful

- ✓ Better debugging, tooling, performance
- ✓ ~20% compiler perf improvement

https://github.com/fsharp/fslang-design/tree/master/FSharp-4.0

F# 4.1

- ✓ Optional large scope cycles more on this later)
- ✓ Result<T,Error> in standard library
- ✓ Unboxed (struct) tuples
- ✓ Unboxed (struct) records
- ✓ Unboxed (struct) unions

- ✓ More bits and pieces

https://github.com/fsharp/fslang-design/tree/master/FSharp-4.1

F# 4.5

✓ Span<T> high perf type-safe non-allocating code
✓ Improved async debugging

✓ More bits and pieces

https://github.com/fsharp/fslang-design/tree/master/FSharp-4.5

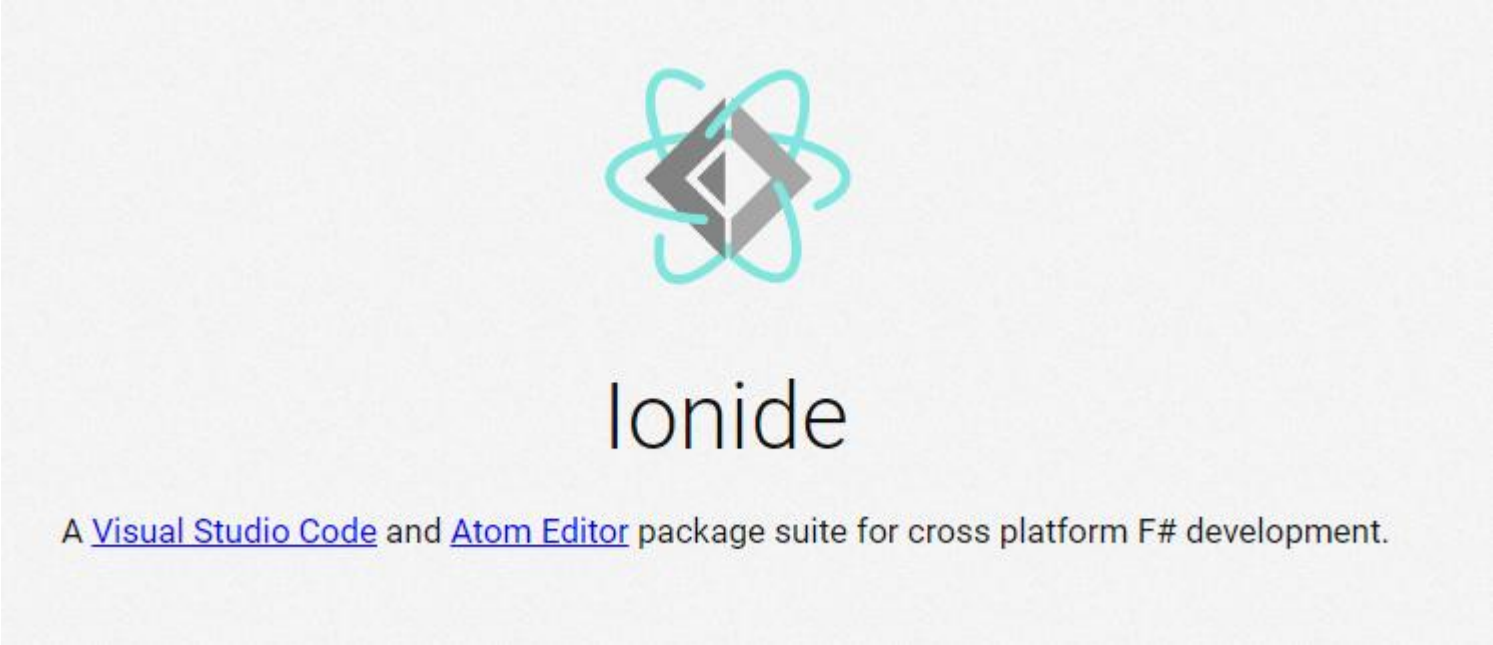
F# 4.6

✓ Anonymous records

✓ More bits and pieces

https://github.com/fsharp/fslang-design/tree/master/FSharp-4.5

VSCode + Ionide = Cross-Platform F# Editing Love



F# and .NET Core (Linux, OSX, Windows)

```
dotnet new -lang F#

dotnet build
```

docs.microsoft.com/dotnet/core/


We're Gonna Build a
Framework!
A song about code by @dylanbeattie
Music "We Didn't Start the Fire" by Billy Joel
© 1989 Columbia Records

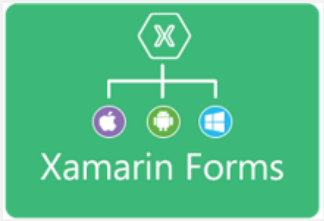
THE CONTEXT

- Initially F# embraced functional-first programming but lacked opinion on functional-first app development
- Fable solves this for **Web**
- But F# devs also need to be able to make **Mobile and Desktop Apps**

SITUATION: THE WORLD OF UI IS DEVELOPING

- OO programming
- Complex mutable models
- Declarative static UI
- View in 2nd markup language
- Complex designer tooling

- Functional programming
- Simple declarative models
- Declarative dynamic UI
- View in same language DSL
- Simple live coding tooling



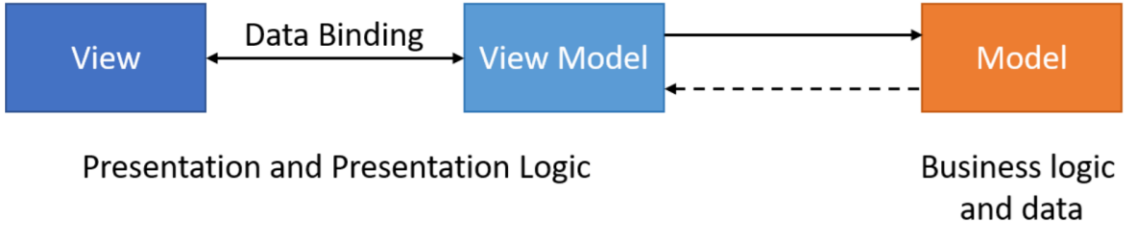
MVVM

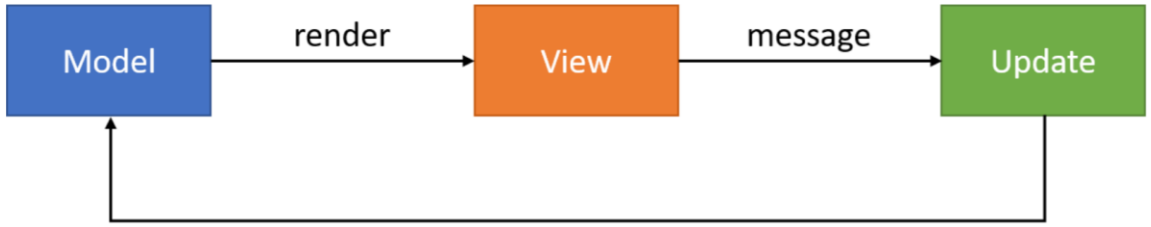
(model-view-viewmodel)

MVU

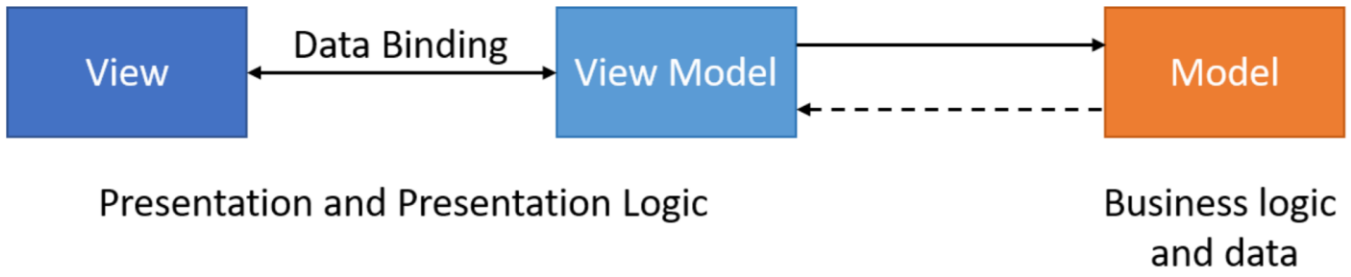
(model-view-update)

MVVM vs MVU


View
Data Binding
View Model
Model
Presentation and Presentation Logic
Business logic
and data


Model
render
View
message
Update

Model View View Model



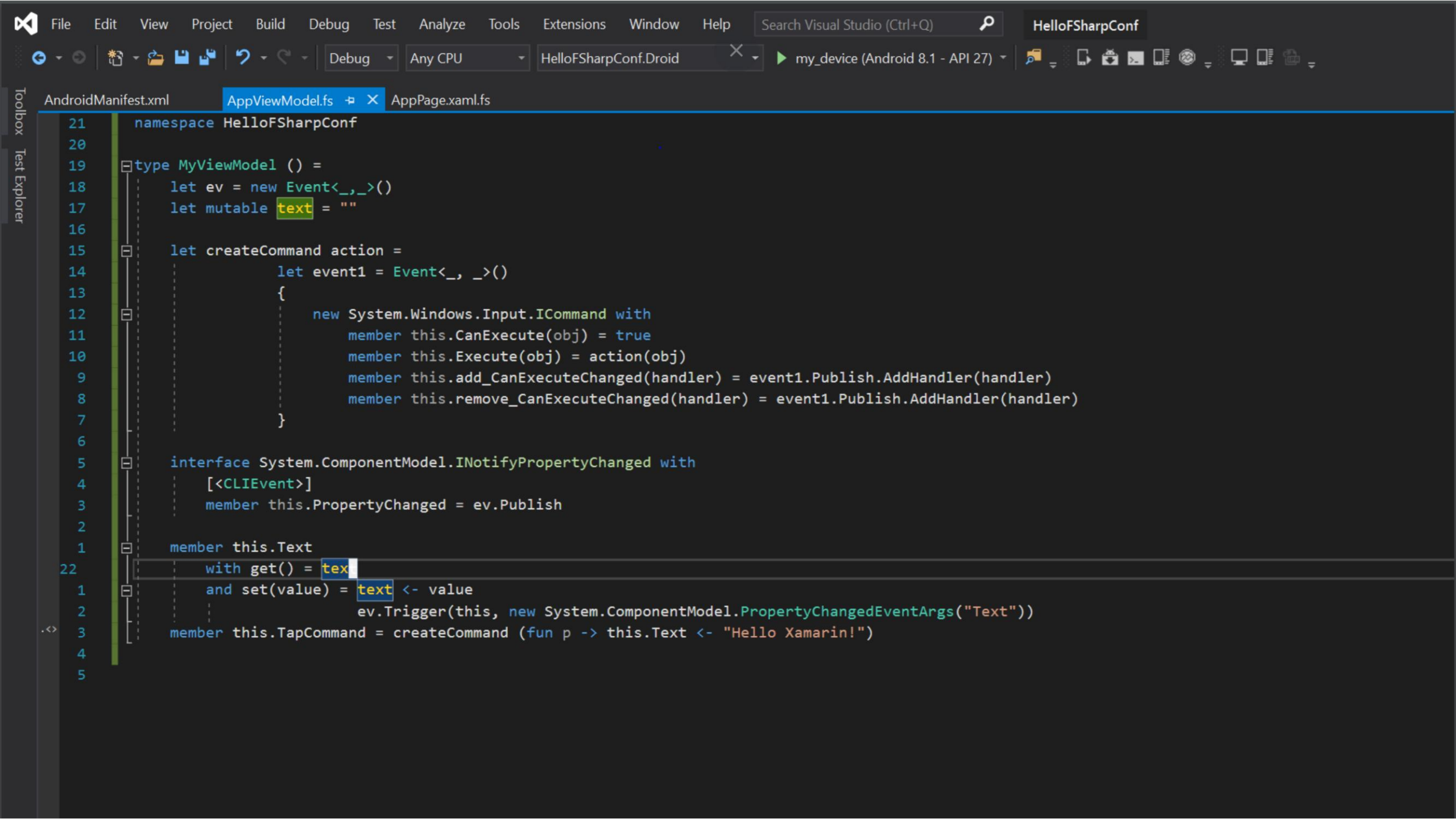
THE VIEW DSL

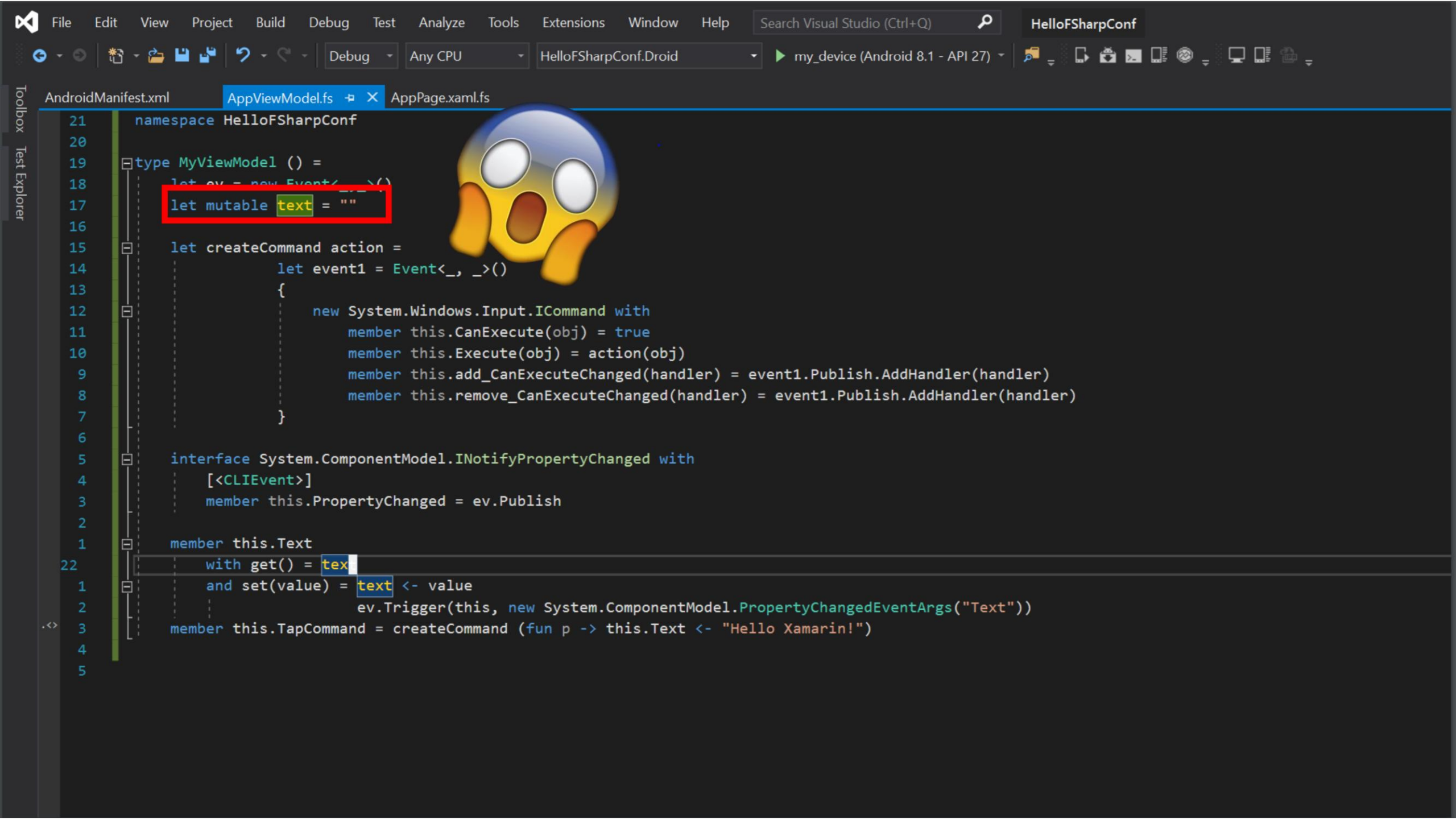
1. Xaml looks like this:

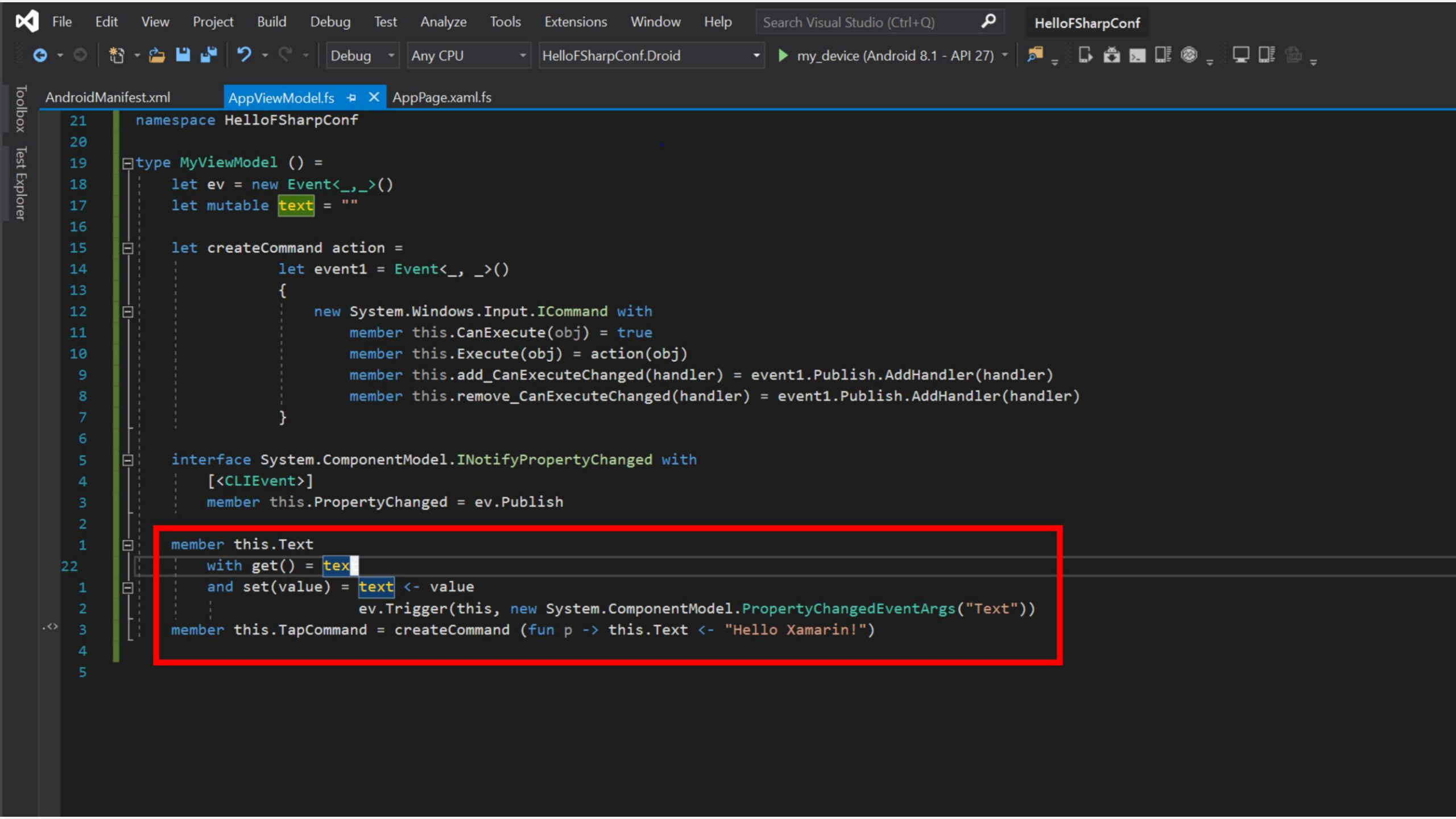
```
<?xml version="1.0" encoding="UTF-8"?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms" xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml" x:C
       <ContentPage.Content>
        <StackLayout Padding="20">
          <StackLayout Padding="20" VerticalOptions="Center">
            <Label Text="{Binding Path=[CounterValue], StringFormat=' value 1: {0}'}" HorizontalOptions="Center" Text
            <Label Text="{Binding Path=[CounterValue2], StringFormat=' value 2: {0}'}" HorizontalOptions="Center" Tex
            <Button Text="Increment" Command="{Binding Path=[IncrementCommand]}"   />
            <Button Text="Decrement" Command="{Binding Path=[DecrementCommand]}" />
            <Slider Maximum="10" Minimum="1" Value="{Binding Path=[StepValue]}" />
            <Label Text="{Binding Path=[StepValue], StringFormat='Step size: {0}'}" HorizontalOptions="Center" />
            <!-- This is included to show how the visibility of controls is controlled -->
        </StackLayout>
        <Button Text="Reset"  IsVisible="{Binding Path=[ResetVisible]}" HorizontalOptions="Center" Command="{Binding
      </StackLayout>
       </ContentPage.Content>
</ContentPage>
```

THE PROBLEM WITH XAML

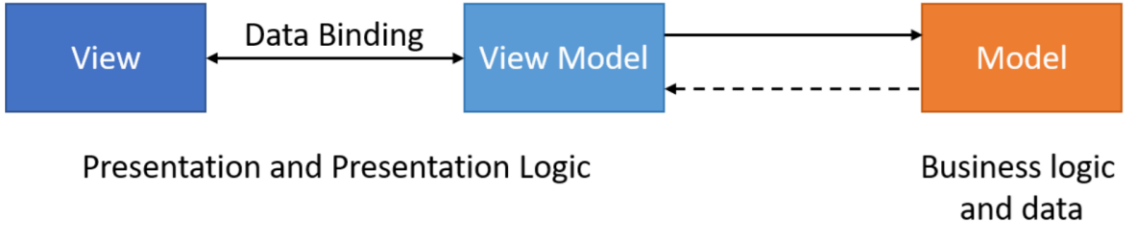
- A core problem: Xaml
  - Xaml assumes "designers" will "sliced off" UI from code into a separate DSL.
  - Xaml is **static**, with **layers of complexity** (e.g. templating), and **code-like constructs** (behaviours, convertors), and requires **complex designer tooling** (which often fails).
  - Xaml stems from a fundamental category mistake in 2003 ("dynamic UI should be separated into a static markup language")

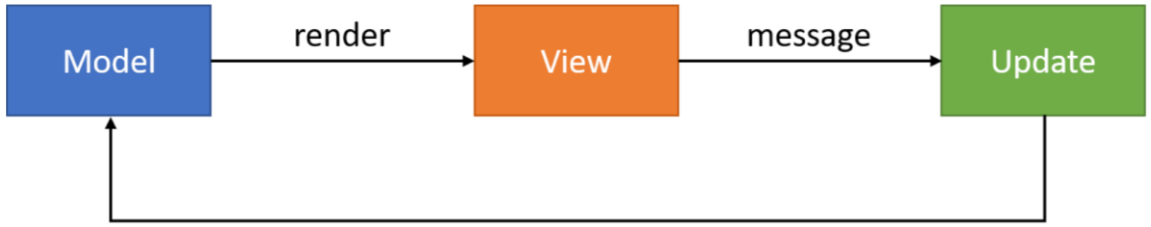

File Edit View Project Build Debug Test Analyze Tools Extensions Window Help
Search Visual Studio (Ctrl+Q)
HelloFSharpConf
Debug
Any CPU
HelloFSharpConf.Droid
my_device (Android 8.1 - API 27)
AndroidManifest.xml
AppViewModel.fs
AppPage.xaml.fs
Toolbox
Test Explorer
namespace HelloFSharpConf
type MyViewModel () =
let ev = new Event<_,_>()
let mutable text = ""
let createCommand action =
let event1 = Event<_, _>()
{
new System.Windows.Input.ICommand with
member this.CanExecute(obj) = true
member this.Execute(obj) = action(obj)
member this.add_CanExecuteChanged(handler) = event1.Publish.AddHandler(handler)
member this.remove_CanExecuteChanged(handler) = event1.Publish.AddHandler(handler)
}
interface System.ComponentModel.INotifyPropertyChanged with
[<CLIEvent>]
member this.PropertyChanged = ev.Publish
member this.Text
with get() = tex
and set(value) = text <- value
ev.Trigger(this, new System.ComponentModel.PropertyChangedEventArgs("Text"))
member this.TapCommand = createCommand (fun p -> this.Text <- "Hello Xamarin!")


File Edit View Project Build Debug Test Analyze Tools Extensions Window Help
Search Visual Studio (Ctrl+Q)
HelloFSharpConf
Debug
Any CPU
HelloFSharpConf.Droid
my_device (Android 8.1 - API 27)
AndroidManifest.xml
AppViewModel.fs
AppPage.xaml.fs
Toolbox
Test Explorer
namespace HelloFSharpConf
type MyViewModel () =
let mutable text = ""
let createCommand action =
let event1 = Event<_, _>()
new System.Windows.Input.ICommand with
member this.CanExecute(obj) = true
member this.Execute(obj) = action(obj)
member this.add_CanExecuteChanged(handler) = event1.Publish.AddHandler(handler)
member this.remove_CanExecuteChanged(handler) = event1.Publish.AddHandler(handler)
interface System.ComponentModel.INotifyPropertyChanged with
[<CLIEvent>]
member this.PropertyChanged = ev.Publish
member this.Text
with get() = tex
and set(value) = text <- value
ev.Trigger(this, new System.ComponentModel.PropertyChangedEventArgs("Text"))
member this.TapCommand = createCommand (fun p -> this.Text <- "Hello Xamarin!")

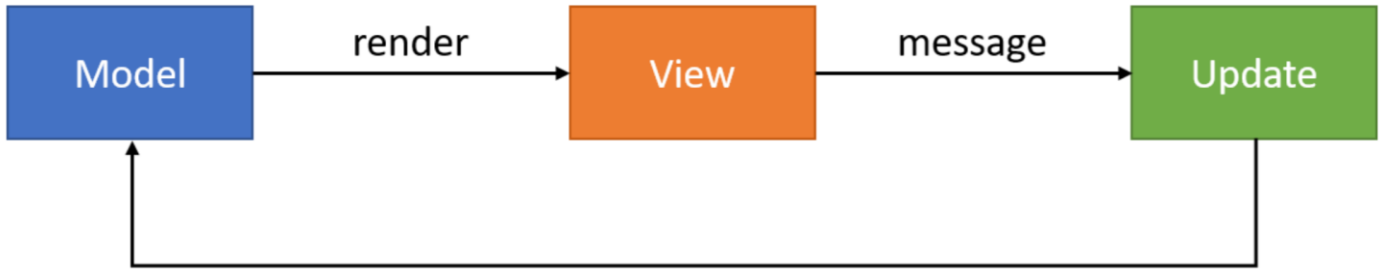

File Edit View Project Build Debug Test Analyze Tools Extensions Window Help
Search Visual Studio (Ctrl+Q)
HelloFSharpConf
Debug
Any CPU
HelloFSharpConf.Droid
my_device (Android 8.1 - API 27)
AndroidManifest.xml
AppViewModel.fs
AppPage.xaml.fs
Toolbox
Test Explorer
namespace HelloFSharpConf
type MyViewModel () =
let ev = new Event<_,_>()
let mutable text = ""
let createCommand action =
let event1 = Event<_, _>()
{
new System.Windows.Input.ICommand with
member this.CanExecute(obj) = true
member this.Execute(obj) = action(obj)
member this.add_CanExecuteChanged(handler) = event1.Publish.AddHandler(handler)
member this.remove_CanExecuteChanged(handler) = event1.Publish.AddHandler(handler)
}
interface System.ComponentModel.INotifyPropertyChanged with
[<CLIEvent>]
member this.PropertyChanged = ev.Publish
member this.Text
with get() = text
and set(value) = text <- value
ev.Trigger(this, new System.ComponentModel.PropertyChangedEventArgs("Text"))
member this.TapCommand = createCommand (fun p -> this.Text <- "Hello Xamarin!")

MVVM vs MVU


View
Data Binding
View Model
Model
Presentation and Presentation Logic
Business logic
and data

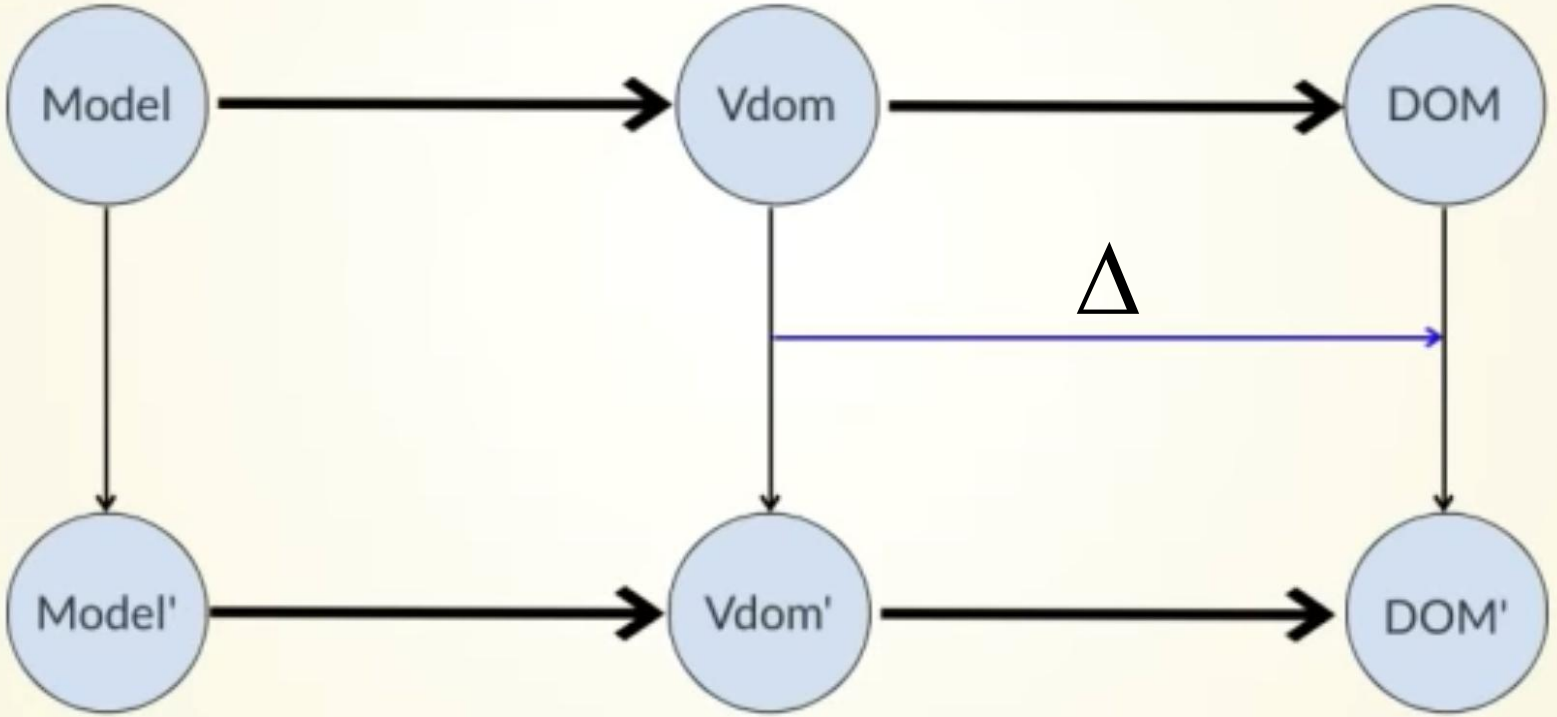

Model
render
View
message
Update

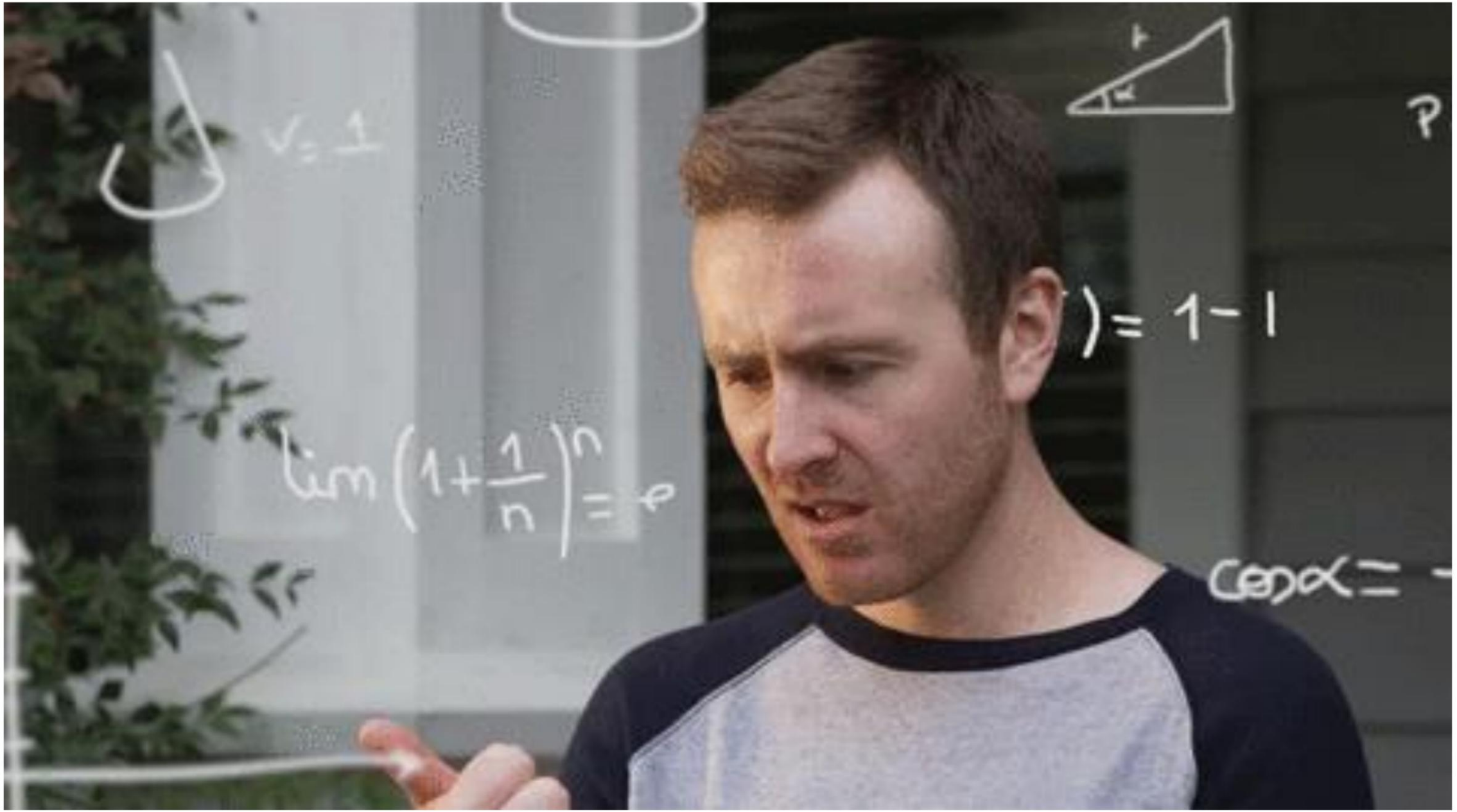
Model View Update



Your UI as an Incremental Computation

Your UI as an Incremental Computation




$\lim\left(1+\frac{1}{n}\right)^n = e$

A SIMPLE MVU APP

```
type Model =
    { … }

type Msg =
    | …
    | …

let init() =
    { … }

let update msg model =
    match msg with
    | … -> { model with … }
    | … -> { model with … }

let view model dispatch =
    if model.IsPressed then
        …  dispatch msg …
    else
        … dispatch msg …
```

The model from which the view is generated

Messages which cause updates to the model

Initial state

Update the model

Compute the view

A SIMPLE MVU APP

```
type Model =
    { IsPressed: bool }

type Msg =
    | Pressed

let init() =
    { IsPressed=false }

let update (msg: Msg) (model: Model) =
    match msg with
    | Pressed -> { model with IsPressed = true }

let view (model: Model) dispatch =
    if model.IsPressed then
        View.Label(text="I was pressed!")
    else
        View.Button(text="Press Me!", command=(fun () -> dispatch Pressed))
```

The model from which the view is generated

Messages which cause updates to the model

Initial state

Update the model

Compute the view

MVU

It's all just functional programming

- UI becomes an information-flow problem
- Views and model update are just functional programming
- Views are **re-computed** and **differentially** applied
- Messages are explicit and easy to trace

THE PROBLEM WITH XAML

how to "!" in a binding in xaml

GeorgeCook

March 2015 edited March 2015 in Xamarin.Forms

i.e.

`

```
<Button Clicked="OnLoginClicked"
    Image="Images/Login/loginWithFacebookButton.png"
    IsEnabled="{!(Binding IsLoggingIn)}"  <<<<<<< this is the bit that I don't know how to do.
```

THE PROBLEM WITH XAML

JoeManke

March 2015

What you want is a converter.

```
<local:NegateBooleanConverter x:Key="inverter"/>
<Button Clicked="OnLoginClicked"
        Image="Images/Login/loginWithFacebookButton.png"
        IsEnabled="{Binding IsLoggingIn, Converter={StaticResource inverter}"/>
```

Then in your code:

```
    public class NegateBooleanConverter : IValueConverter
    {
        public object Convert (object value, Type targetType, object parameter, System.Globalization.CultureInfo cultu
        {
            return !(bool)value;
        }
        public object ConvertBack (object value, Type targetType, object parameter, System.Globalization.CultureInfo c
        {
            return !(bool)value;
        }
```

Good answer but someone who is not much familiar with Xaml they need to add the name space in xaml file also to use that converter

THE VIEW DSL

1. View looks like this:

```
let view (model: Model) dispatch =

  View.ContentPage(content=

    View.StackLayout(padding=30.0,verticalOptions = LayoutOptions.Center, children=[

        View.Label(text= string model.Count, horizontalOptions=LayoutOptions.Center)

        View.Button(text="Increment", command= (fun () -> dispatch Increment))

        View.Button(text="Decrement", command= (fun () -> dispatch Decrement))

        View.Slider(minimum=0.0, maximum=10.0, value=model.Step,
             valueChanged= (fun args -> dispatch (SetStep (int args.NewValue))))

        View.Label(text=sprintf "Step size: %d" model.Step, horizontalOptions=LayoutOptions.Center)

        View.Button(text="Reset", horizontalOptions=LayoutOptions.Center,
             command=(fun () -> dispatch Reset), canExecute = model.Changed)
    ]))
```

THE VIEW DSL

1. View looks like this:

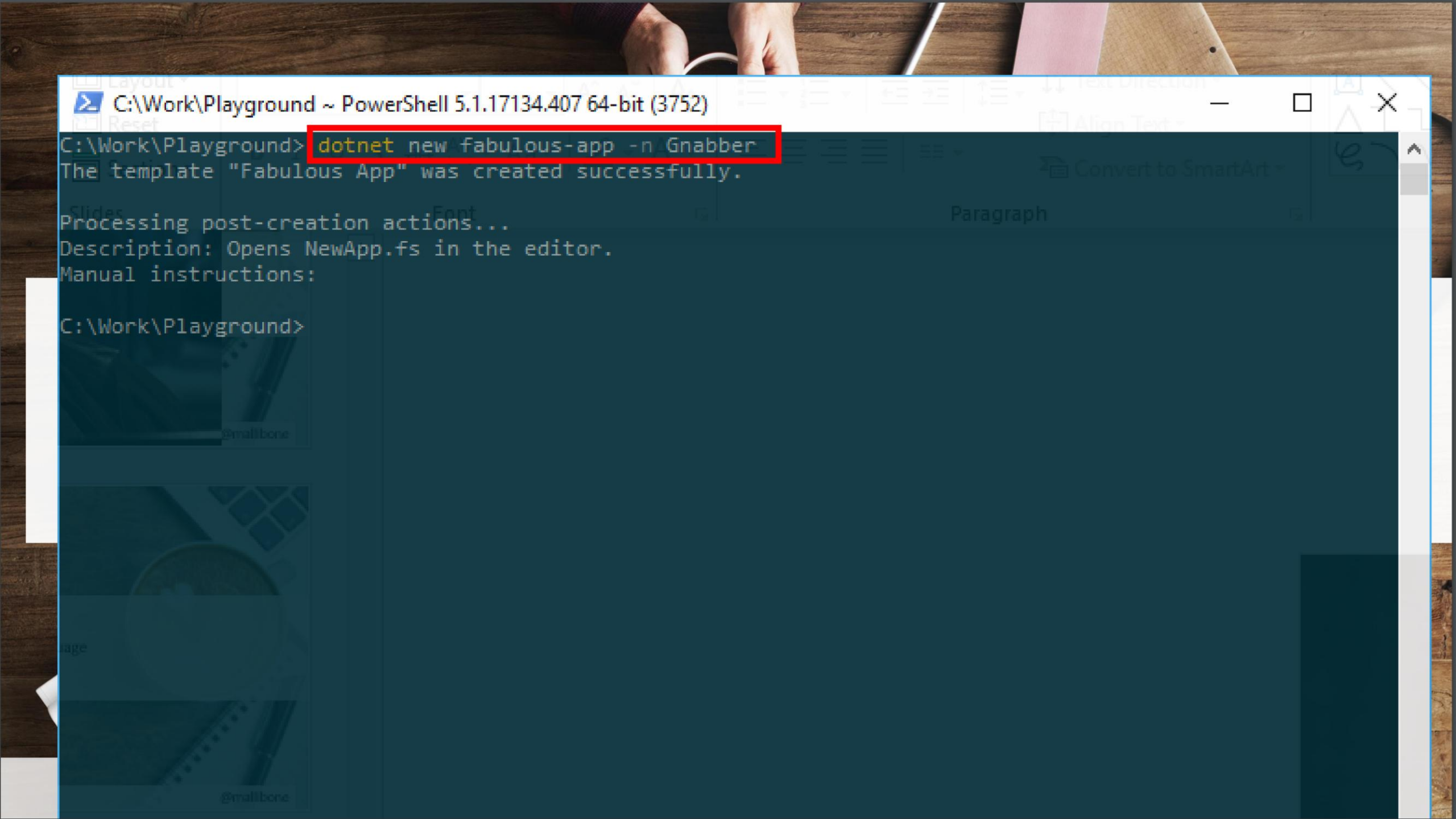
```
let view (model: Model) dispatch =

  View.ContentPage(content=

    View.StackLayout(padding=30.0,verticalOptions = LayoutOptions.Center, children=[

        View.Label(text= string model.Count, horizontalOptions=LayoutOptions.Center)

        View.Button(text="Increment", command= (fun () -> dispatch Increment))

        View.Button(text="Decrement", command= (fun () -> dispatch Decrement))

        View.Slider(minimum=0.0, maximum=10.0, value=model.Step,
             valueChanged= (fun args -> dispatch (SetStep (int args.NewValue))))

        View.Label(text=sprintf "Step size: %d" model.Step, horizontalOptions=LayoutOptions.Center)

        View.Button(text="Reset", horizontalOptions=LayoutOptions.Center,
             command=(fun () -> dispatch Reset), canExecute = not model.Unchanged)
    ]))
```

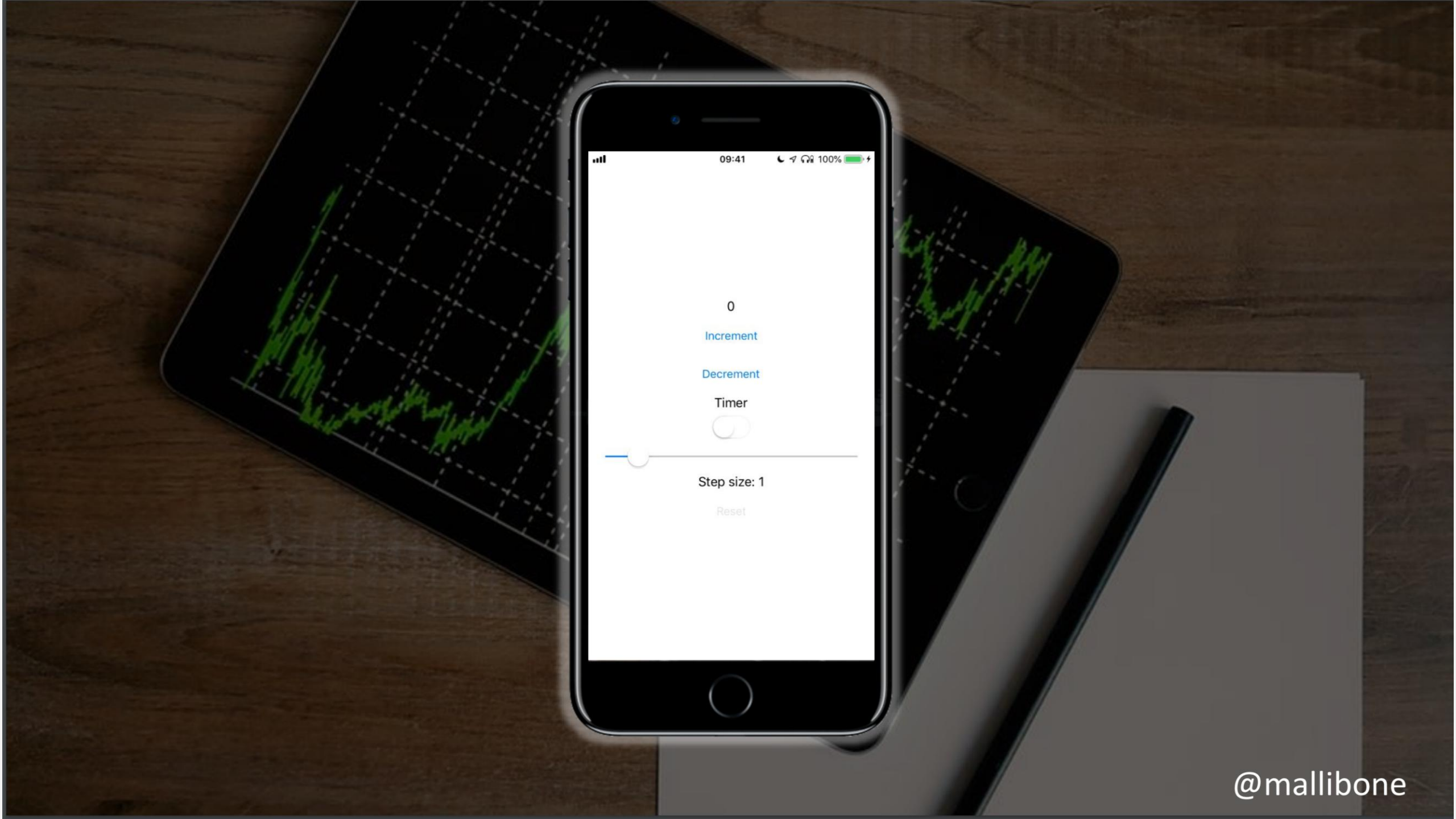
How easy is that?

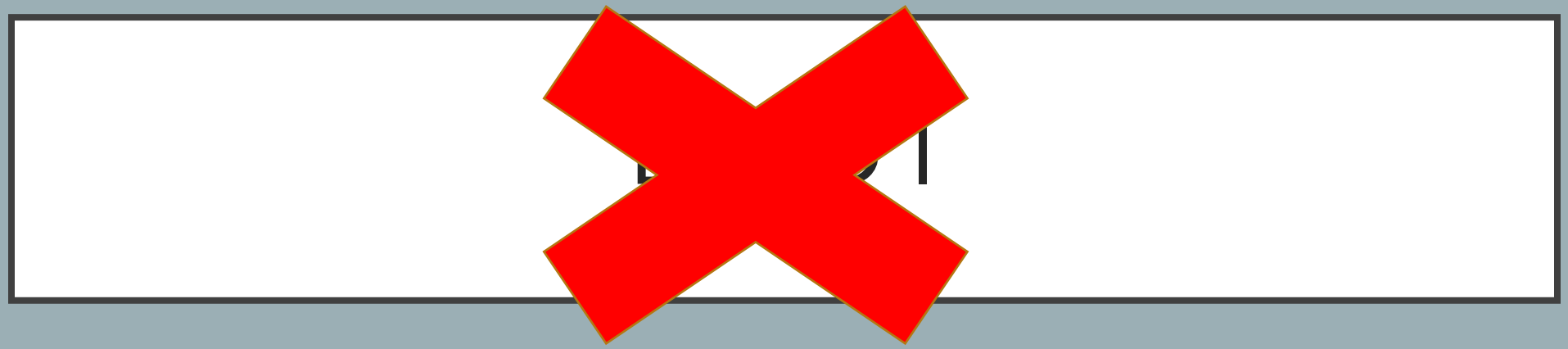
THE NEED

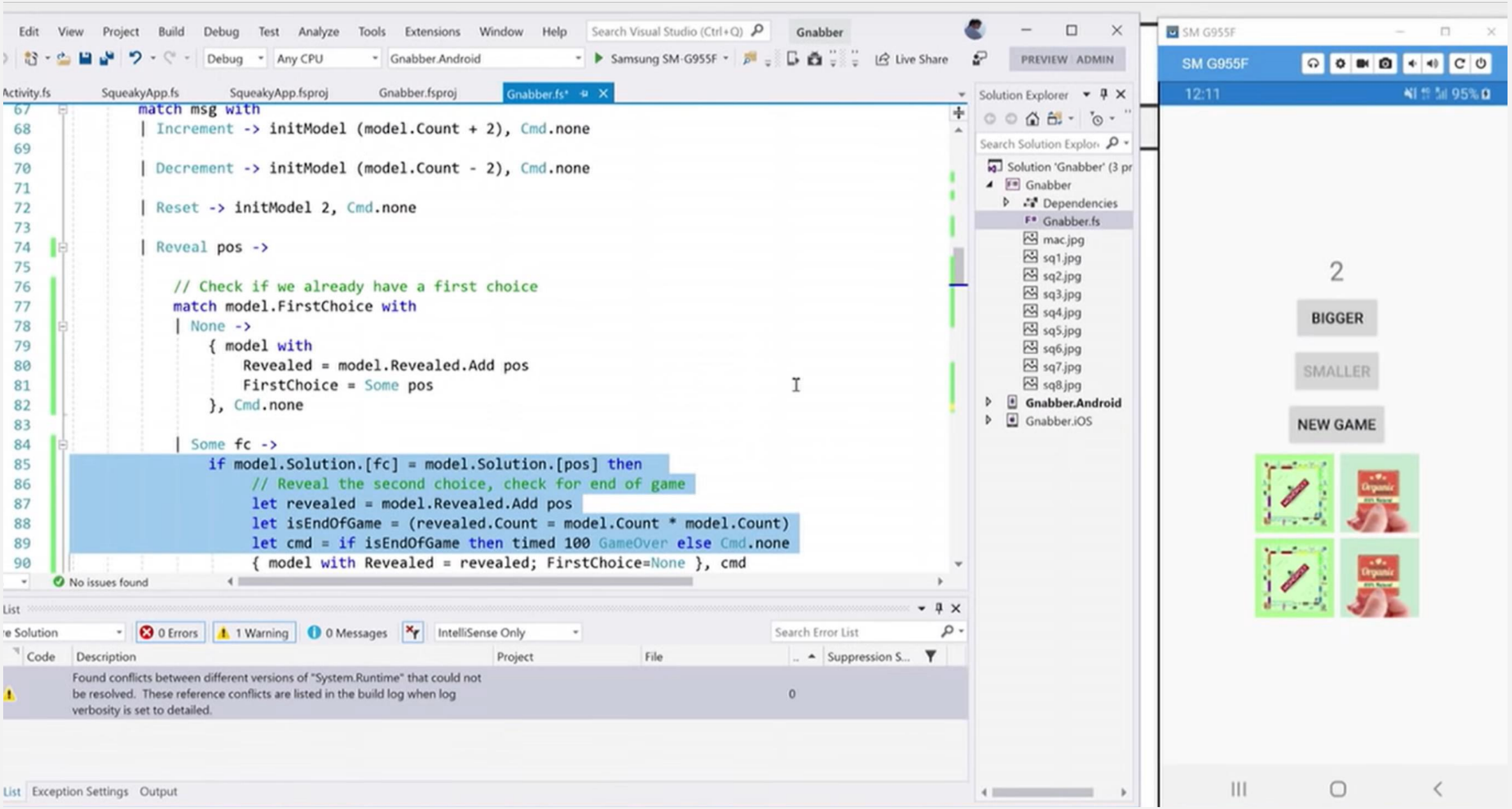
What's needed?

the simplicity of MVU, but for Mobile and Desktop Apps


C:\Work\Playground ~ PowerShell 5.1.17134.407 64-bit (3752)
C:\Work\Playground> dotnet new fabulous-app -n Gnabber
The template "Fabulous App" was created successfully.
Processing post-creation actions...
Description: Opens NewApp.fs in the editor.
Manual instructions:
C:\Work\Playground>


09:41
100%
0
Increment
Decrement
Timer
Step size: 1
Reset
@mallibone

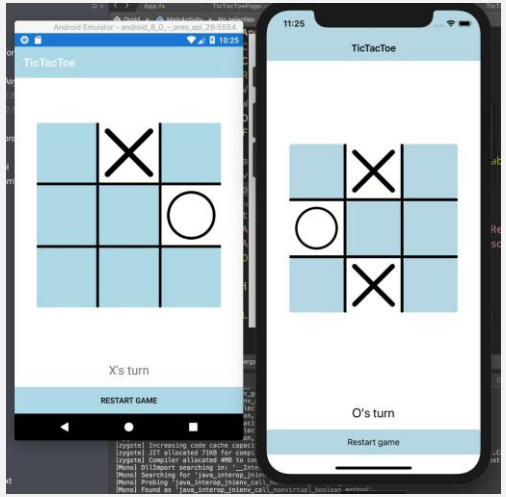



Edit View Project Build Debug Test Analyze Tools Extensions Window Help
Search Visual Studio (Ctrl+Q)
Gnabber
Debug
Any CPU
Gnabber.Android
Samsung SM-G955F
Live Share
PREVIEW ADMIN
Activity.fs
SqueakyApp.fs
SqueakyApp.fsproj
Gnabber.fsproj
Gnabber.fs*
match msg with
| Increment -> initModel (model.Count + 2), Cmd.none
| Decrement -> initModel (model.Count - 2), Cmd.none
| Reset -> initModel 2, Cmd.none
| Reveal pos ->
// Check if we already have a first choice
match model.FirstChoice with
| None ->
{ model with
Revealed = model.Revealed.Add pos
FirstChoice = Some pos
}, Cmd.none
| Some fc ->
if model.Solution.[fc] = model.Solution.[pos] then
// Reveal the second choice, check for end of game
let revealed = model.Revealed.Add pos
let isEndOfGame = (revealed.Count = model.Count * model.Count)
let cmd = if isEndOfGame then timed 100 GameOver else Cmd.none
{ model with Revealed = revealed; FirstChoice=None }, cmd
No issues found
Solution Explorer
Search Solution Explor
Solution 'Gnabber' (3 pr
Gnabber
Dependencies
Gnabber.fs
mac.jpg
sq1.jpg
sq2.jpg
sq3.jpg
sq4.jpg
sq5.jpg
sq6.jpg
sq7.jpg
sq8.jpg
Gnabber.Android
Gnabber.iOS
List
0 Errors
1 Warning
0 Messages
IntelliSense Only
Search Error List
Code
Description
Project
File
Suppression S...
Found conflicts between different versions of "System.Runtime" that could not be resolved. These reference conflicts are listed in the build log when log verbosity is set to detailed.
0
List
Exception Settings
Output
SM G955F
SM G955F
12:11
95%
2
BIGGER
SMALLER
NEW GAME

@mallibone

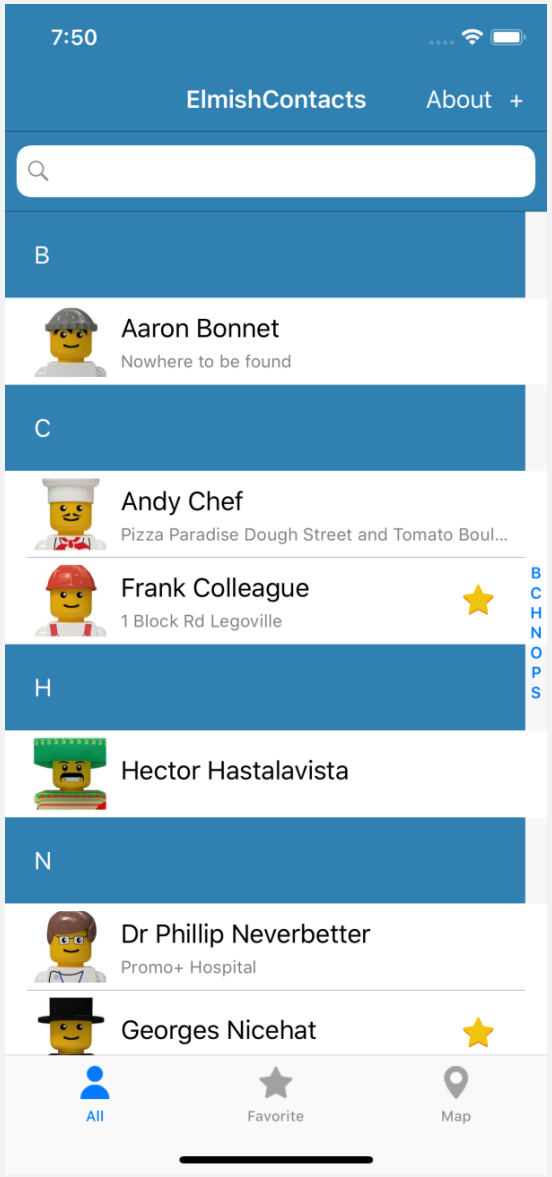
Doing it with style

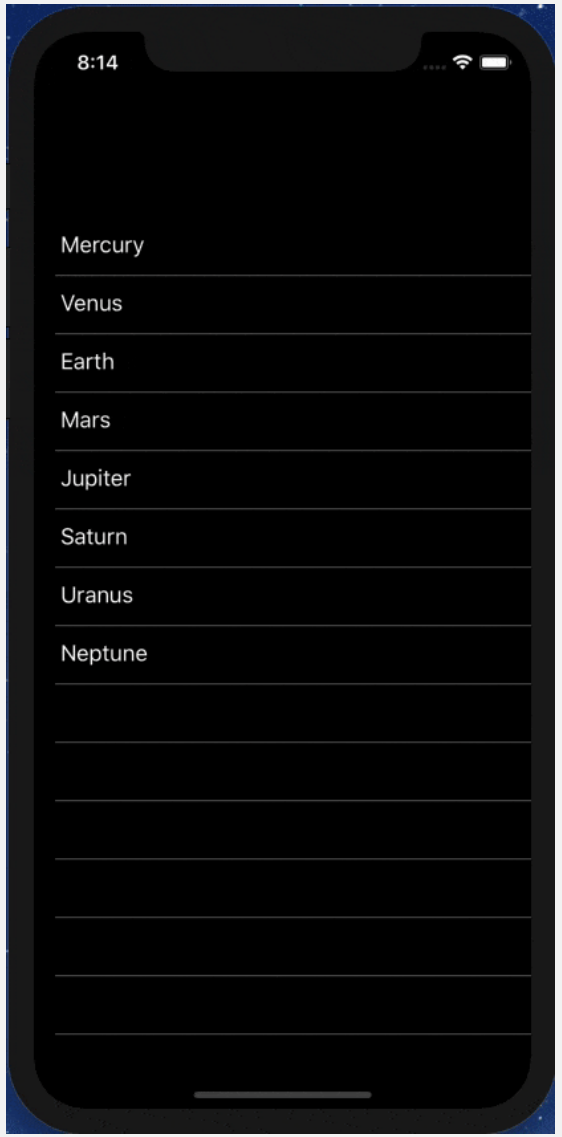
@mallibone

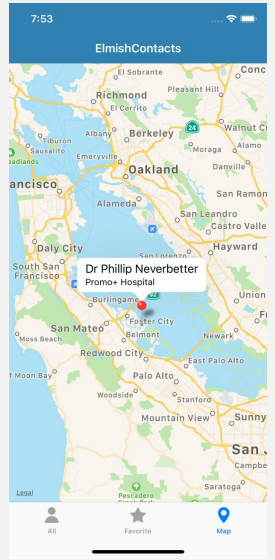


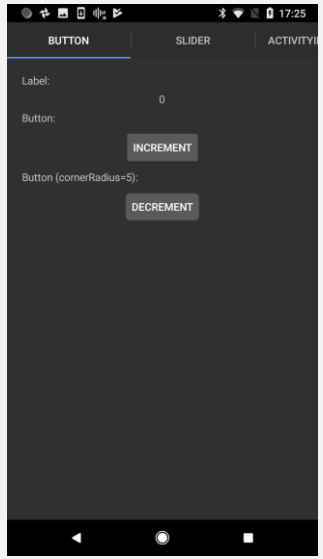
GET IT ON Google Play

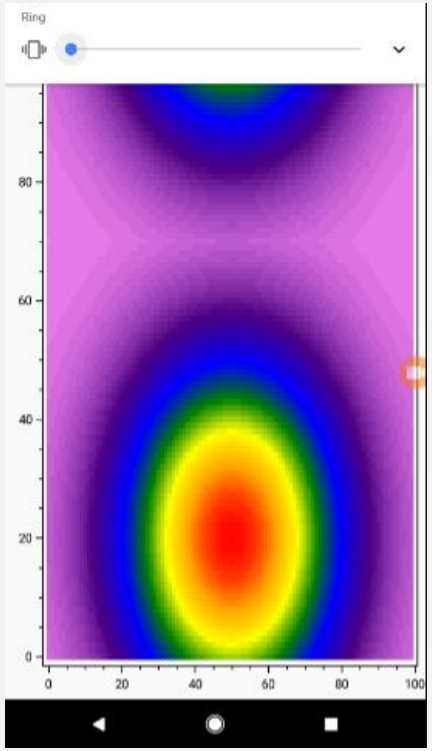
Download on the App Store



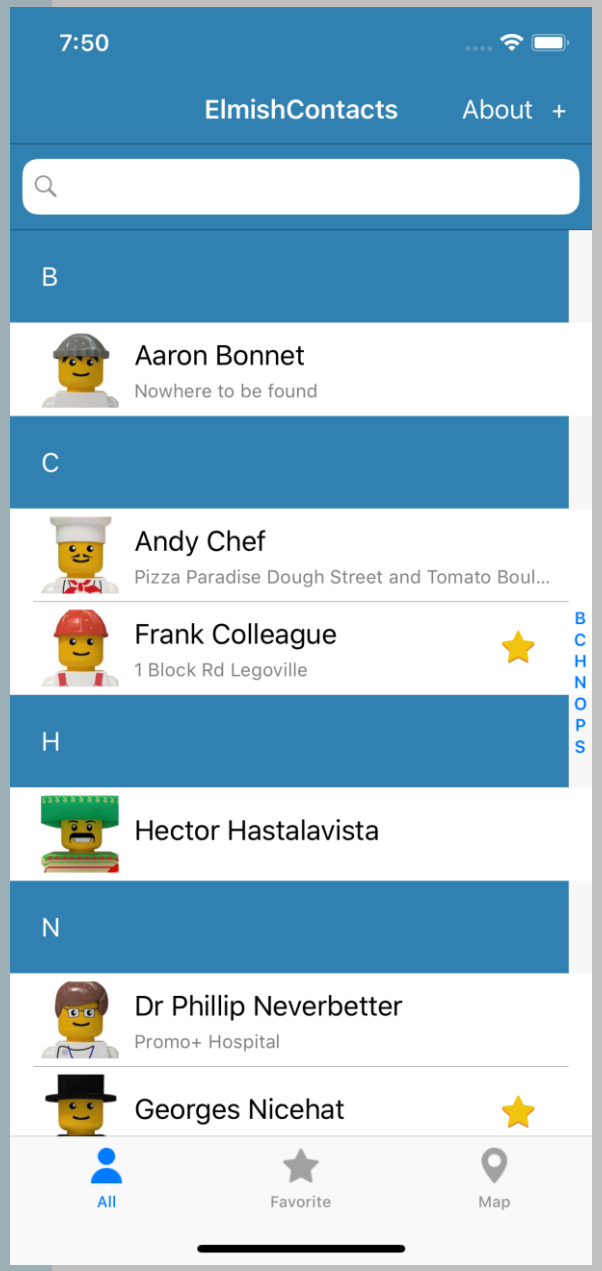


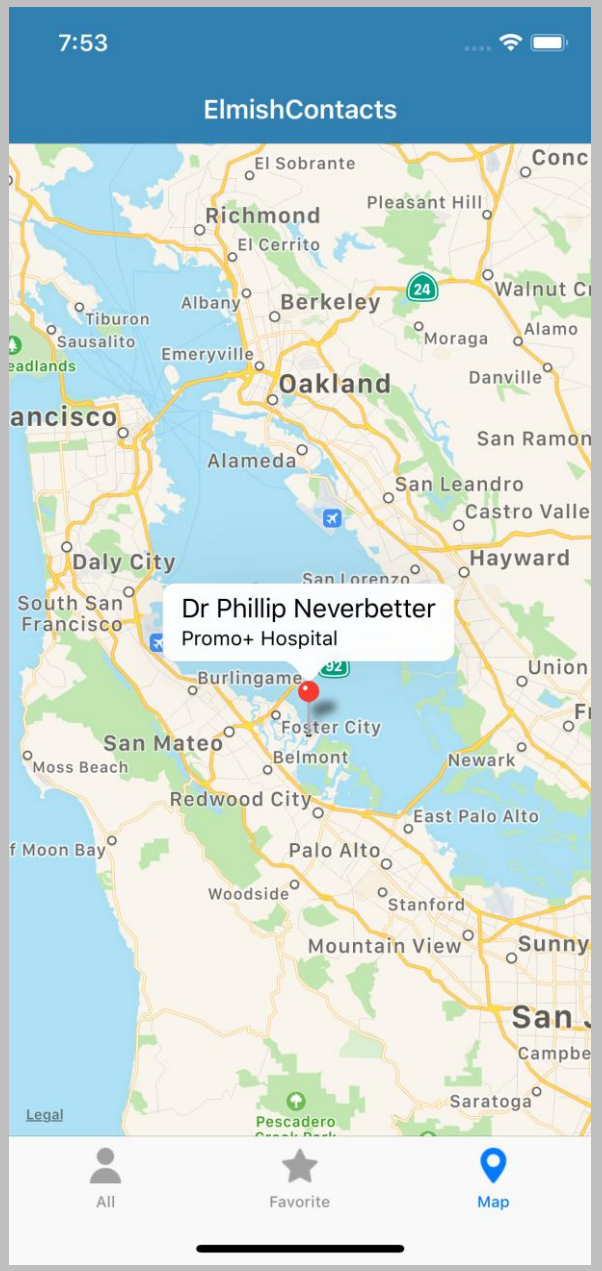






ELMISH CONTACTS





LEARN!

ElmishContacts is a great example

- On-device per-app SQL-Lite integration
- Using maps
- Asynchronous data loading onto maps
- Catching exceptions and being robust
- Sending SMS, making phone calls and Email using Xamarin.Essentials
- Multi-page with external messages reconciled via the composed app
- Editing and searching lists of things
- Taking photos
- Splash screens
- Decent design

THE VIEW DSL

1. Xaml looks like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms" xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml" x:C
       <ContentPage.Content>
        <StackLayout Padding="20">
          <StackLayout Padding="20" VerticalOptions="Center">
            <Label Text="{Binding Path=[CounterValue], StringFormat=' value 1: {0}'}" HorizontalOptions="Center" Text
            <Label Text="{Binding Path=[CounterValue2], StringFormat=' value 2: {0}'}" HorizontalOptions="Center" Tex
            <Button Text="Increment" Command="{Binding Path=[IncrementCommand]}"   />
            <Button Text="Decrement" Command="{Binding Path=[DecrementCommand]}" />
            <Slider Maximum="10" Minimum="1" Value="{Binding Path=[StepValue]}" />
            <Label Text="{Binding Path=[StepValue], StringFormat='Step size: {0}'}" HorizontalOptions="Center" />
            <!-- This is included to show how the visibility of controls is controlled -->
        </StackLayout>
        <Button Text="Reset"  IsVisible="{Binding Path=[ResetVisible]}" HorizontalOptions="Center" Command="{Binding
      </StackLayout>
       </ContentPage.Content>
</ContentPage>
```

THE VIEW DSL

1. View looks like this:

```
let view (model: Model) dispatch =

  View.ContentPage(content=

    View.StackLayout(padding=30.0,verticalOptions = LayoutOptions.Center, children=[

        View.Label(text= string model.Count, horizontalOptions=LayoutOptions.Center)

        View.Button(text="Increment", command= (fun () -> dispatch Increment))

        View.Button(text="Decrement", command= (fun () -> dispatch Decrement))

        View.Slider(minimum=0.0, maximum=10.0, value=model.Step,
             valueChanged= (fun args -> dispatch (SetStep (int args.NewValue))))

        View.Label(text=sprintf "Step size: %d" model.Step, horizontalOptions=LayoutOptions.Center)

        View.Button(text="Reset", horizontalOptions=LayoutOptions.Center,
             command=(fun () -> dispatch Reset), canExecute = model.Changed)
    ]))
```

ANIMATIONS

- Yes you can
- Uses “refs” like React

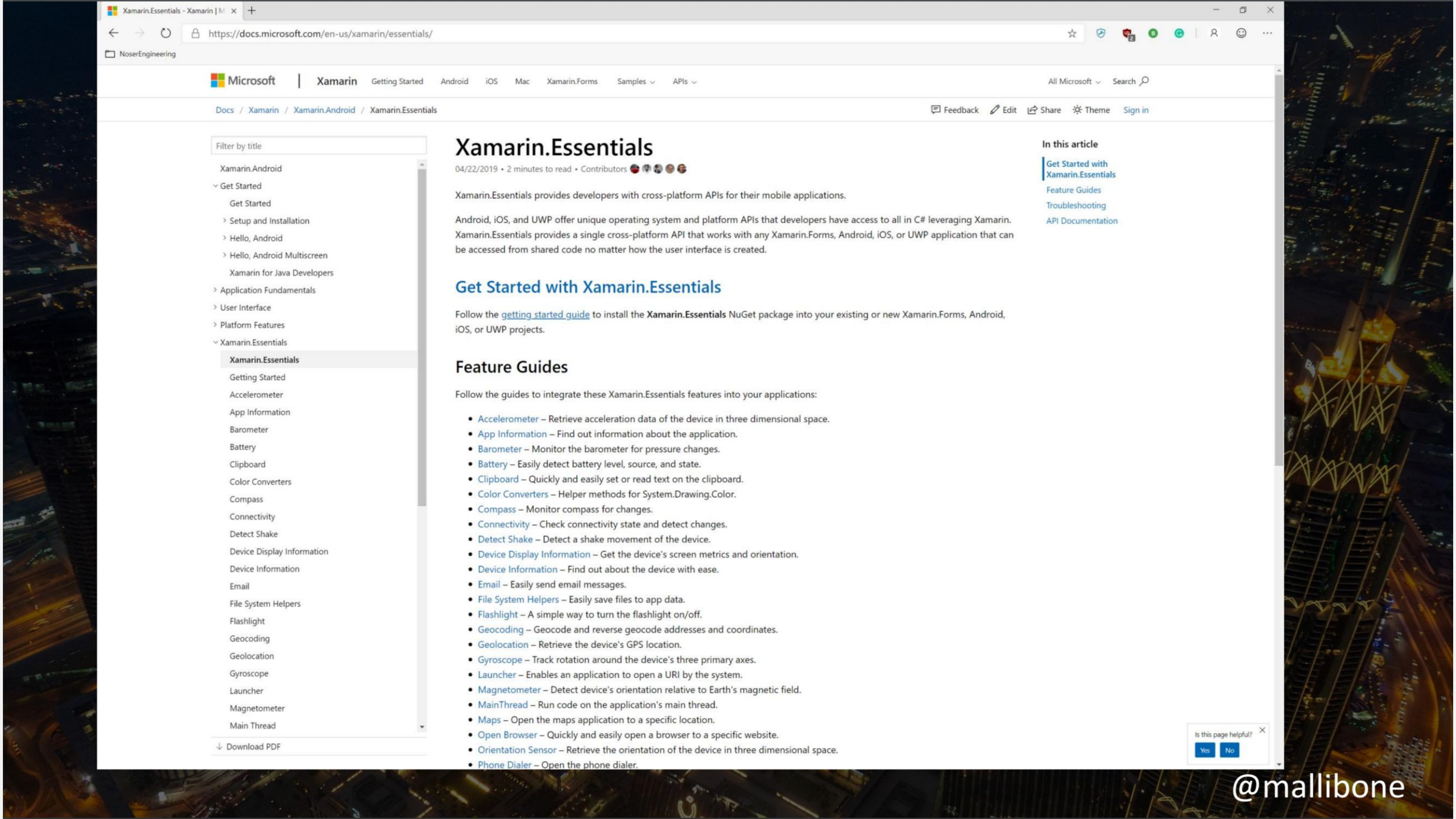
```
let animatedLabelRef = ViewRef<Label>()

let update msg model =
    match msg with
    | Poked ->
        match animatedLabelRef.TryValue with
        | None -> ()
        | Some c -> c.RotateTo (360.0, 2000u) |> ignore

let view dispatch model =
    View.StackLayout [
        View.Label(text="Rotate", ref=animatedLabelRef)
        View.Button(text="Rotate", command=(fun () -> dispatch Poked))
    ]
```

MULTI-PAGE APPS

- Yes you can
- Still a single view function
  - CarouselPage, TabbedPage, NavigationPage with child pages
- Each page has a model, message type and update function
- You compose the pieces using F# code

PRAGMATICS

- State persistence
- Time-travel debugging
- App size
- Platform-specifics
- Deploying
- Extensibility


Xamarin.Essentials - Xamarin | M
https://docs.microsoft.com/en-us/xamarin/essentials/
NoserEngineering
Microsoft | Xamarin Getting Started Android iOS Mac Xamarin.Forms Samples APIs
All Microsoft Search
Docs / Xamarin / Xamarin.Android / Xamarin.Essentials
Feedback Edit Share Theme Sign in
Filter by title
Xamarin.Android
Get Started
Get Started
Setup and Installation
Hello, Android
Hello, Android Multiscreen
Xamarin for Java Developers
Application Fundamentals
User Interface
Platform Features
Xamarin.Essentials
Xamarin.Essentials
Getting Started
Accelerometer
App Information
Barometer
Battery
Clipboard
Color Converters
Compass
Connectivity
Detect Shake
Device Display Information
Device Information
Email
File System Helpers
Flashlight
Geocoding
Geolocation
Gyroscope
Launcher
Magnetometer
Main Thread
Download PDF
Xamarin.Essentials
04/22/2019 • 2 minutes to read • Contributors
Xamarin.Essentials provides developers with cross-platform APIs for their mobile applications.
Android, iOS, and UWP offer unique operating system and platform APIs that developers have access to all in C# leveraging Xamarin. Xamarin.Essentials provides a single cross-platform API that works with any Xamarin.Forms, Android, iOS, or UWP application that can be accessed from shared code no matter how the user interface is created.
Get Started with Xamarin.Essentials
Follow the getting started guide to install the Xamarin.Essentials NuGet package into your existing or new Xamarin.Forms, Android, iOS, or UWP projects.
Feature Guides
Follow the guides to integrate these Xamarin.Essentials features into your applications:
Accelerometer – Retrieve acceleration data of the device in three dimensional space.
App Information – Find out information about the application.
Barometer – Monitor the barometer for pressure changes.
Battery – Easily detect battery level, source, and state.
Clipboard – Quickly and easily set or read text on the clipboard.
Color Converters – Helper methods for System.Drawing.Color.
Compass – Monitor compass for changes.
Connectivity – Check connectivity state and detect changes.
Detect Shake – Detect a shake movement of the device.
Device Display Information – Get the device's screen metrics and orientation.
Device Information – Find out about the device with ease.
Email – Easily send email messages.
File System Helpers – Easily save files to app data.
Flashlight – A simple way to turn the flashlight on/off.
Geocoding – Geocode and reverse geocode addresses and coordinates.
Geolocation – Retrieve the device's GPS location.
Gyroscope – Track rotation around the device's three primary axes.
Launcher – Enables an application to open a URI by the system.
Magnetometer – Detect device's orientation relative to Earth's magnetic field.
MainThread – Run code on the application's main thread.
Maps – Open the maps application to a specific location.
Open Browser – Quickly and easily open a browser to a specific website.
Orientation Sensor – Retrieve the orientation of the device in three dimensional space.
Phone Dialer – Open the phone dialer.
In this article
Get Started with Xamarin.Essentials
Feature Guides
Troubleshooting
API Documentation
Is this page helpful?
Yes
No

@mallibone

MAKING IT EFFICIENT (HALF)

```
let view model dispatch =
    …
    View.Slider(minimum=0.0, maximum=10.0,
        value=double model.Count,
        valueChanged=(fun args -> dispatch (SliderValueChanged args.NewValue)))
    …
```

This is a tree of immutable data describing the view

We want to avoid reallocating this data and re-traversing on each model update

MAKING IT EFFICIENT (HALF)

```
let view model dispatch =
    …
    dependsOn (…) (fun model (count, step) ->
        …
    )
    …
```

MAKING IT EFFICIENT (HALF)

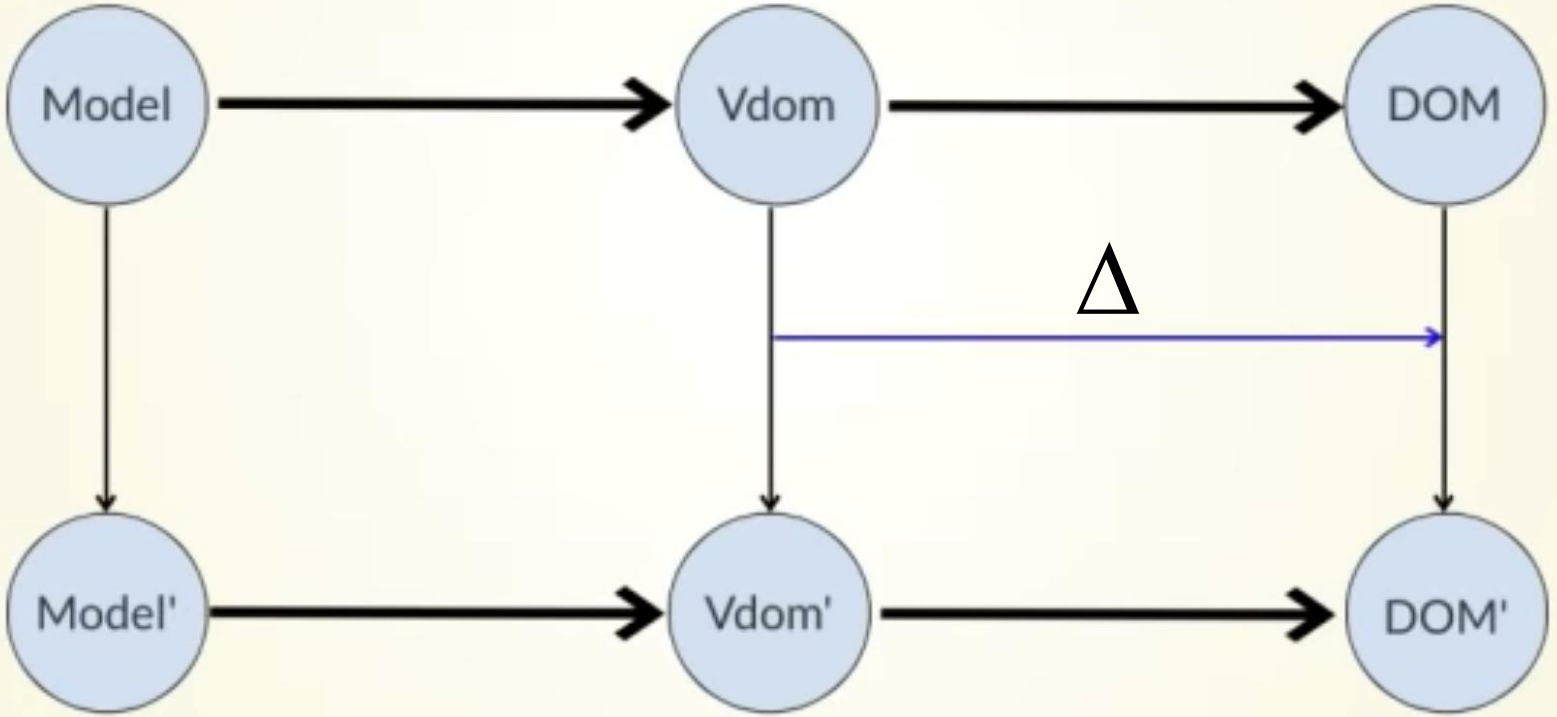
```
let view model dispatch =
    …
    dependsOn (model.Count, model.Step) (fun model (count, step) ->
        View.Slider(minimum=0.0, maximum=10.0,
            value=double count,
            valueChanged=(fun arg -> dispatch (SliderValueChanged args.NewValue))))
    …
```

This recycles subtrees that are unchanged. Unchanged subtrees are not diff'd

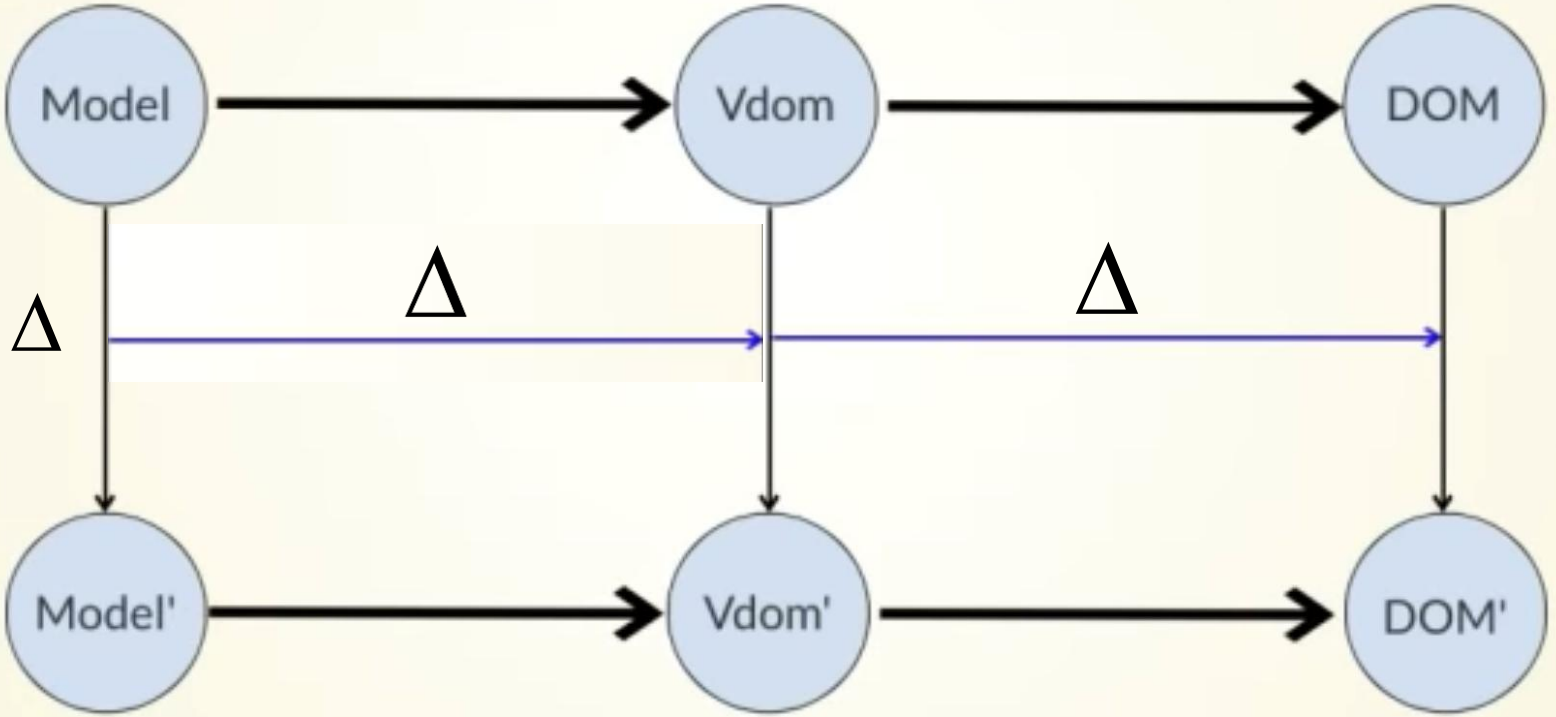
MAKING IT EFFICIENT (OTHER HALF)

- This is good, and enough for many information apps
- We can do better, and it will be the focus of future work
- Please come and help us apply Self-Adjusting Computations (incremental immutable data structures) to Fabulous
- This is an invention of the PL community, and already applied in OCaml

Your UI as an Incremental Computation



Your UI as an Incremental Computation



POSSIBLE WINDOW DRESSING

```
type Model =
    { … }

type Msg =
    | …
    | …

let init() =
    { … }

let update msg model = …

let view model dispatch = …

let app = mkApp (init, update, view)
```

POSSIBLE WINDOW DRESSING

```
[<ReflectedDefinition>]
type Model =
    { … }

type Msg =
    | …
    | …

let init() =
    { … }

let update msg model = …

let view model dispatch = …

let app = mkIncrementalApp (init, update, view)
```

Incrementalize the app,
(using F# quotations)

Summary

F# Emphasises Clear, Code to Solve Real-world Problems

MVU makes FP relevant to UI

Fabulous brings this to cross-platform apps

R&D opportunities to apply incremental immutable data (SAC) for high perf

Questions?